Customer App (Frontend): Developer 2 Handover

Purpose and scope

The customer app lets a customer pick a gate, pay for items, and watch a scooter reach that gate within the promised time. Developer 2 owns the whole customer app.

NOTE: These features,screens,architecture etc all the things are decided by the information shared with the product scope if it is changed then the endpoints and some things might change…

In scope

- Every customer screen: sign-in, gate setup, availability, cart and payment, finding a scooter, countdown, handover, no-scooter options, history, help.
- The data layer that calls the backend, the live order channel, and the local cache.
- Payment and push integration on the phone side.
- Tests and release builds for Android and iOS.

Out of scope for this doc

- Design and visual styling (the designer's track). Developer 2 styles screens from the design tokens only.
- Backend, dispatch and the tracker service (Developer 1).
- Rider app and partner app (separate docs).
- DigiLocker, DLT SMS registration and FSSAI (later legal items; use test OTP until then).

Customer app

Feature	What the person does or sees	Why it exists
Sign in	Phone number and OTP	Safe accounts without passwords
Gate setup	Picks a map pin, names the gate, adds entry notes, saves it	The promise is to a gate, so it must be exact
Nearby availability	Sees only items truly in reach, with quantity left	No false promises
Cart and payment	Adds items and pays; the payment is held, then taken at acceptance	Nobody is charged if no scooter comes
Finding a scooter	A waiting screen with a clear timeout	The wait is shown, not hidden
Accepted countdown	90 s timer, rough scooter position, masked call	The core promise
At the gate	Meet your rider; OTP handover	Proof of delivery

Feature	What the person does or sees	Why it exists
No scooter available	Longer ETA, notify me, or free cancel	The customer is never trapped
History and reorder	Past orders, receipts, one-tap reorder	Repeat orders
Help	Report missing or cold food; refund path	Trust
Notifications	Push and SMS at each state	Updates when the app is closed

Words used in these docs

Term	Plain meaning
Gate	The exact entry point the customer picks for delivery
Gate pin	The gate's map location plus entry instructions
Sweet spot	The area a scooter can start from and still reach a gate within the time budget
Staging node	A legal place where a scooter waits for orders
Lot	One labelled batch of food with its prepared and expiry times
Reservation	A short hold on units so nobody else can buy them
Acceptance	The moment the server confirms the order and starts the 90 s clock
Leg	One scooter's share of an order
Hub	A small site where scooters restock
Kill switch	A control that turns the 90 s offer off for a zone
p90	The time that 9 out of 10 trips beat

Backend needed before you start

Developer 1 delivers these in week 1. Until the real backend exists, build against the mock server.

1. OpenAPI contract

- Covers every endpoint in the API section below, including OTP endpoints.
- Every response includes server_time (used for the countdown).
- Every repeatable write requires an Idempotency-Key header.

2. Mock server (Prism or equivalent)

- Answers every endpoint with schema-valid data.
- Scenario header X-Mock-Scenario selects the state to return.
- Scenarios the app must handle: no scooter available; offer timeout; offer accepted; payment declined; payment captured; order handed over; socket drop and reconnect; last unit taken by another order; gate not verified; zone with the 90-second offer switched off.

3. Live channel mock

- A WebSocket that replays the status sequence for the chosen scenario, with server_time on each event.

4. Fixtures

- Test gates, menu items, and availability in the pilot area.
- A test customer account with test OTP numbers.

5. Environments

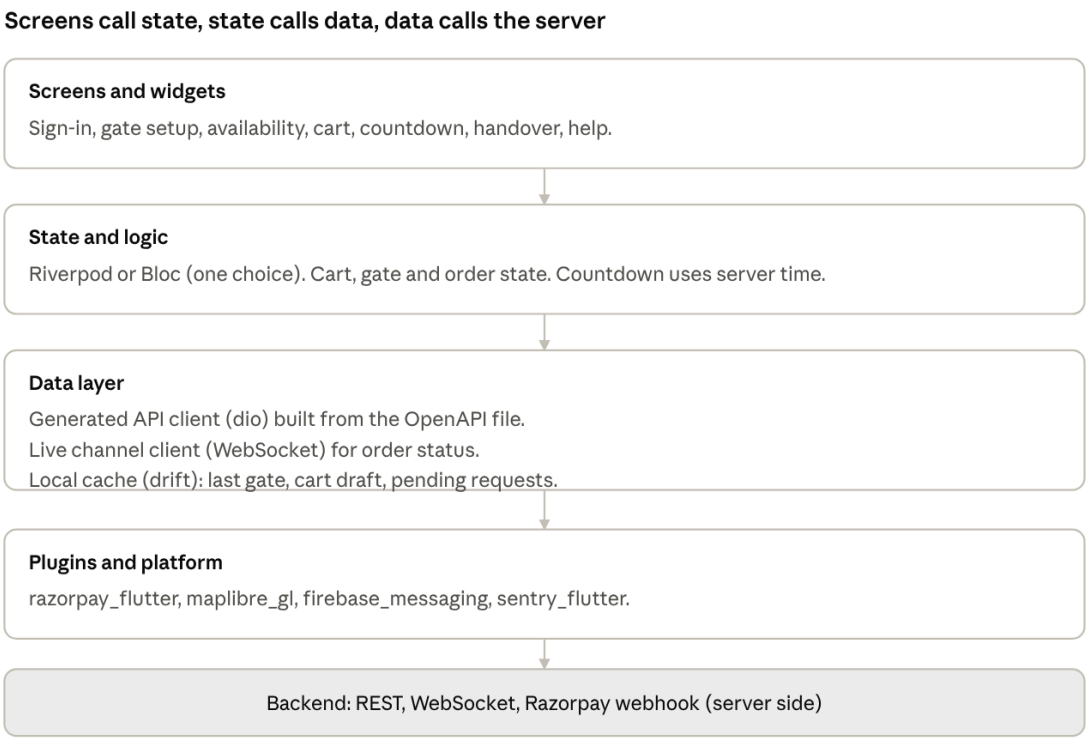
- Mock URL for development, stable for the build.
- Staging URL with Razorpay test keys, available from Phase 2.

What to do while waiting

- Build sign-in, gate setup and availability on the mock.
- Build the countdown and order screens on the live-channel mock.
- Real payment is only tested after Staging is ready.

- And all the screens given by UI/UX

Architecture



customer app layers · 4 layers, one server

Screens never call the server directly. Every request goes through the data layer.

Tech stack and libraries

Flutter is decided. Items marked Verify must be checked on Android and iOS before use.

Need	Choice	Status
Framework	Flutter, one version pinned for the team	Decided
Language	Dart	Decided
State	Riverpod or Bloc (pick one in week 1), Bloc Recommended	Open
Routing	go_router	Proposed
HTTP	dio	Proposed
API client	Generated from OpenAPI (openapi-generator, Dart dio)	Proposed
Live channel	web_socket_channel	Proposed
Local cache	drift (SQLite)	Proposed
Models	freezed, json_serializable	Proposed
Payments	razorpay_flutter	Verify
Maps	Maplibre_gl , google maps	Verify
Push	firebase_messaging (FCM)	Proposed
Crash reports	sentry_flutter	Proposed
Secure token storage	flutter_secure_storage	Proposed
Native code	Only where a plugin needs it: Kotlin (Android), Swift (iOS)	Only if needed

Rules

- Pin the Flutter and Dart versions in the repo.
- One state library for the whole app.
- No direct HTTP calls from screens; every call goes through the data layer.

Tools

- Source and CI: GitHub with GitHub Actions. Runs analyze, unit tests, widget tests and the build on every pull request.
- Mock and contract: Prism mock server; Schemathesis contract tests against the OpenAPI file.
- API manual testing: Bruno or Postman, with the same collection the team shares.

- Design handoff: Figma, read-only for Developer 2; tokens exported as files.
- Test builds: TestFlight (iOS) and Google Play internal track (Android).
- Crash and performance: Sentry for crashes and slow frames.
- Task tracking: the team's chosen tracker (Jira or Linear; pick one). - optional
- Device testing: two budget Android phones and one current iPhone, all on throttled networks.

Screens and tasks

Each screen lists its data, the states it must handle, and when it is done. Every screen also needs loading, empty and error states.

C1. Project setup

- Navigation, theme from tokens, generated API client, environment switch (mock, staging, production).
- Done when: the app starts on the mock and shows a placeholder home.

C2. Welcome and OTP sign-in

- Data: POST /v1/auth/otp/request; POST /v1/auth/otp/verify; POST /v1/auth/refresh.
- States: wrong code; resend limit reached; no network; code expired.
- Done when: a customer signs in on the mock and the token survives an app restart.

C3. Gate setup

- Data: GET /v1/gates; POST /v1/gates.
- Pick a pin on the map, name the gate, add entry notes.
- States: no location permission; pin outside the service area; duplicate gate.
- Done when: a saved gate is still there after reinstall.

C4. Nearby availability and item detail

- Data: GET /v1/availability?gate_id=; GET /v1/menu/items/{id}.
- Shows only items in reach, with quantity left.
- States: nothing available; stale data (refresh); item sold out while viewing.
- Done when: the list updates when stock changes on the mock.

C5. Cart and payment

- Data: POST /v1/orders with Idempotency-Key; Razorpay checkout from the response.
- States: declined card; double tap; app sent to background during payment.
- Done when: one tap creates exactly one order, even on a slow network.

C6. Finding a scooter

- Data: WS /v1/orders/{id}/stream.
- Shows a waiting screen with a clear timeout and a cancel option.
- States: timeout; app closed and reopened; cancel.
- Done when: the waiting state and its timeout show correctly after a restart.

C7. Accepted countdown

- Data: the same order stream, with accepted time and en route.
- Counts down from the server accepted time, corrected for phone clock offset.

- States: clock offset; reconnect; target missed (shown calmly, see the rules below).
- Done when: the countdown survives an app restart and a dropped connection.

C8. At the gate

- Data: arrival event from the stream; handover OTP.
- States: rider late; wrong OTP; no network during handover.
- Done when: handover is confirmed once, even if the request is sent twice.

C9. No scooter available

- Data: order state with no offer.
- Options: wait, switch, cancel free.
- Done when: each option works and the payment hold is released on cancel.

C10. Order history and reorder

- Data: GET /v1/orders; GET /v1/orders/{id}.
- Done when: past orders list correctly and reorder fills the cart.

C11. Help and refund status

- Data: POST /v1/support/tickets; GET /v1/support/tickets/{id}; GET /v1/orders/{id}/refund.
- Done when: a customer reports a problem and sees the refund status.

C12. Push and deep links

- Tapping a notification opens the right order screen, showing the current state (fetched first), not the state in the message.
- Done when: each notification type opens the correct screen.

C13. Offline and reconnect

- Shows a connection status bar.
- On reconnect, fetches the current order state before showing anything.
- Done when: the app recovers after airplane mode without a stale screen.

C14. Localisation, accessibility, themes

- Text scales with system settings; screen-reader labels on every control; light and dark themes.
- Done when: the checklist in the testing section passes.

C15. Native bridges (only if a plugin needs them)

- Payment and push bridges in Kotlin and Swift.
- Done when: the plugin cannot do the job without it, and the change is documented.

NOTE : ONLY the Main screens are mentioned not the sub-screens,overlays,pop-ups,ALL ENDPOINTS or any other optional screens or things that will be developed by developer with co-ordination of UI/UX designer accordingly .

API endpoints the app calls

All paths are under /v1. Every request sends Authorization: Bearer <token>, except the OTP endpoints. Every response includes server_time. Error responses use one shape: code, message, and retryable (true or false).

Area	Method and path	Request key fields	Response key fields	Used in
Auth	POST /v1/auth/otp/request	phone	request_id, expires_in_s	C2
Auth	POST /v1/auth/otp/verify	request_id, otp	access_token, refresh_token, customer_id	C2
Auth	POST /v1/auth/refresh	refresh_token	access_token, refresh_token	all
Gates	GET /v1/gates	none	gates[] (id, label, pin, status, verified)	C3
Gates	POST /v1/gates	label, lat, lng, entry_notes	gate (id, status)	C3
Menu	GET /v1/menu/items/{id}	none	item (name, price, allergens, photo_url)	C4
Availability	GET /v1/availability?gate_id=	gate_id	items[] (item_id, qty_left, stale_at)	C4
Orders	POST /v1/orders (Idempotency-Key)	gate_id, items[] (item_id, qty)	order (id, state), payment (razorpay_order_id, amount, currency)	C5
Orders	GET /v1/orders/{id}	none	order (state, accepted_at, server_time)	C6–C9
Orders	POST /v1/orders/{id}/cancel	reason	order (state)	C9
Orders	GET /v1/orders	cursor, limit	orders[], next_cursor	C10
Live	WS /v1/orders/{id}/stream	token in first frame	events: state, accepted_at, eta_source, server_time	C6–C8
Handover	POST /v1/orders/{id}/handover-confirm	otp	order (state)	C8
Support	POST /v1/support/tickets	order_id, type, text	ticket (id, status)	C11
Support	GET /v1/support/tickets/{id}	none	ticket (status, messages[])	C11

Area	Method and path	Request key fields	Response key fields	Used in
Refund	GET /v1/orders/{id}/refund	none	refund (status, expected_by)	C11

Notes

- eta_source is measured, placeholder or unavailable. The app shows the time only for measured; placeholder and unavailable show the honest wording from the copy deck.
- The Razorpay payment itself is confirmed on the server through the webhook. The app never marks an order paid from its own result.
- Endpoints added after the earlier draft: OTP details, handover-confirm, refund status. Developer 1 adds them in week 1.
- Field names are a starting point. Developer 1 and Developer 2 agree final names in the first contract review and record them in the contract.

Real-time and payment rules

These rules protect the honesty of the timing. They are not optional.

1. **Countdown uses server time.** Show the accepted_at from the server plus 90 seconds, corrected by the measured difference between the phone clock and server_time. Never start a timer on the phone.
2. **Show a time only when the server says it is measured.** If eta_source is placeholder or unavailable, show the copy deck's honest wording instead of a countdown.
3. **Reconnect safely.** When the socket drops, reconnect with a growing delay, then fetch the order state before showing anything.
4. **A notification opens the latest state**, not the state written in the message.
5. **One tap, one order.** Send an Idempotency-Key with every create; a repeated tap or retry must not create a second order.
6. **The app never marks an order paid.** Payment is confirmed by the server through the Razorpay webhook. The app shows pending until the server says captured or authorised.
7. **Cancel before acceptance releases the hold.** The app says "no payment taken; any hold releases within days", never "nobody is charged".

Testing and release checklist

No build goes to testers until every item is ticked.

- ☐ Unit tests for the data layer, state and countdown maths pass.
- ☐ Widget tests for every screen state in the screen list pass.
- ☐ Integration tests against the mock server pass in CI.
- ☐ Contract tests against the Developer 1 OpenAPI file pass.
- ☐ Countdown stays within one second of server time on two budget Android phones, on a throttled network.
- ☐ Socket drop and reconnect recovers without a stale screen.
- ☐ One tap creates exactly one order (tested with a repeated tap).
- ☐ Light and dark themes checked on every screen.

- ☐ Text scales with system settings without clipping.
- ☐ Screen-reader labels on every control.
- ☐ Local-language strings fit without clipping.
- ☐ Permissions asked only when needed, with a plain reason.
- ☐ No new crash in the last build in Sentry.
- ☐ Build shipped to internal testers before any real customer.

Open points

These are not decided yet. Developer 2 can make the choice listed, record it in the doc, and tell Developer 1 if it changes an endpoint.

- State library: Riverpod or Bloc. Choose in week 1; do not switch later.
- Final field names in each API response. Agree in the first contract review.
- Exact wording of each honest-time message. The copy deck owns the words; the app shows them.
- Whether a native bridge is needed for payment or push. Decide only if the plugin fails the test.
- Whether iOS is in the pilot. If Android only, iOS builds are internal test builds only.
- Map provider details (MapLibre tile source and style) once the pilot map extract exists.
- Localisation languages for the pilot city.

If a feature or flow changes during the build, record the change in this doc with the date, the reason, and the developer who made it.